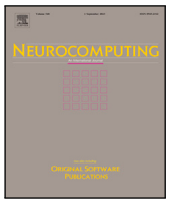
Neurocomputing 638 (2025) 130138



Contents lists available at ScienceDirect

Neurocomputing

journal homepage: www.elsevier.com/locate/neucom





An evolutionary metaheuristic for forming teams in the classroom with constraints

Gonzalo Candel [d], Victor Sánchez-Anguix [b,*], Juan M. Alberola [a], Vicente Julián [a], Vicent Botti [a,c]

[a] Valencian Research Institute for Artificial Intelligence, Universitat Politècnica de València, Camino de Vera, s/n, Valencia, 46022, Spain
[b] Instituto Tecnológico de Informática, Grupo de Sistemas de Optimización Aplicada, Universitat Politècnica de València, Camino de Vera, s/n, Valencia, 46022, Spain
[c] Valencian Graduate School and Research Network of Artificial Intelligence, Cami de Vera, s/n, Valencia, 46022, Spain
[d] Universitat Politècnica de València, Cami de Vera, s/n, Valencia, 46022, Spain

ARTICLE INFO

Keywords:
Team formation
Artificial intelligence
Metaheuristics
Evolutionary algorithm
Teamwork
Classroom

ABSTRACT

Team formation is essential for developing teamwork-related skills in educational settings. The problem of team formation in the classroom consists of partitioning a classroom into non-overlapping teams of students, including every single student. Several algorithms have been proposed to automate the formation of teams, each employing different criteria for guiding the team formation process. Traditionally, metaheuristics have been a common approach due to the combinatorial complexity of the problem. This paper introduces a novel and general evolutionary algorithm for team formation in the classroom guided by mutation, the general concept of synergy between team members, and local search. Our algorithm allows for flexible team size constraints and the inclusion of compulsory and forbidden student combinations, which are not considered in existing methods but are important for capturing human relationships in the classroom. In addition, our algorithm is independent of the specific objective function employed to evaluate the teams formed. We present experiments comparing our proposal with other state-of-the-art algorithms, demonstrating robust performance across different objective functions employed in the team formation literature, superior scalability as the problem size increases, and remarkable performance in settings with or without the aforementioned constraints.

1. Introduction

Teamwork is one of the key elements in the daily operations of many companies to carry out different projects that require the participation of several individuals. In this regard, developing teamwork-related skills is already being implemented in higher education degrees. It is widely accepted that, apart from technical skills, it is necessary to develop these transversal skills in classroom environments [1]. Among others, this allows students to develop communication, leadership, conflict resolution, and time management skills and engage in cooperative learning [2].

To promote the development of these teamwork-related skills, team formation is a crucial phase to allow the team members to interact with each other to solve a task [3]. In this sense, it is important to identify patterns that allow for the successful formation of teams so that the experience is satisfactory [4].

The formation of optimal teams is a topic that has attracted the scientific community's interest. This is a highly combinatorial problem, where many variants are NP-hard problems [5], making it unfeasible to find the optimal solution when instance sizes scale up. The problem of team formation in the classroom consists of dividing students into different and non-overlapping teams based on various criteria that define the optimality of the division. In our previous work [6], we presented a mixed integer linear programming model that allows us to solve the team formation in the classroom problem when different realistic constraints are present. More specifically, one can define flexible team sizes, compulsory combinations of students, and forbidden combinations of students. One of the insights from our previous work is that solvers struggle to solve relatively large problem instances in a reasonable time.

Due to this complexity, several studies have employed approximate algorithms to tackle the team formation in the classroom problem [7–13]. Some of these metaheuristic algorithms have been employed to form teams according to well-known theories such as Belbin's role taxonomy [4,10,14], the Myers-Briggs type indicator [15,16], the Big Five

* Corresponding author.
E-mail address: vsanchez@eio.upv.es (V. Sánchez-Anguix).

https://doi.org/10.1016/j.neucom.2025.130138
Received 4 June 2024; Received in revised form 31 January 2025; Accepted 24 March 2025
Available online 9 April 2025
0925-2312/© 2025 The Authors. Published by Elsevier B.V. This is an open access article under the CC BY-NC license (http://creativecommons.org/licenses/by-nc/4.0/).

inventory [9,17], academic performance [13], class sociograms [12], students' competency levels to perform different tasks [11], or gender diversity [7], among others.

However, these proposals have significant limitations when considering team cardinality and student assignment constraints. On the one hand, few proposals support defining a range for team sizes, so previous solutions limit all teams to a fixed size or limit the number of teams created. On the other hand, no proposals allow defining student assignment constraints, that is, students who must be on the same team and students who should not be on the same team. In real practice, teachers often incorporate these types of constraints, which capture social aspects, forming groups manually [15].

In this article, we present a general metaheuristic algorithm for the team formation problem in the classroom that overcomes the aforementioned limitations. This solution is based on an evolutionary algorithm for team formation in the classroom guided by mutation, the general concept of synergy between team members, and local search to enhance the solutions. Our proposal incorporates constraints when defining the different allowed team sizes and compulsory and forbidden student combinations. Moreover, the model is not constrained by any specific objective function, allowing the utilization of several functions applied to team formation problems. This leads us to outline different goals for our research: (i) to demonstrate that our proposal performs well in scenarios with no special constraints compared to other state-of-the-art proposals; (ii) to demonstrate that our method achieves a good performance in scenarios with constraints related to team size as well as compulsory and forbidden combinations of students; (iii) to demonstrate that our proposal maintains robust performance using different objective functions widely employed in the literature to assess the quality of a team partition; and (iv) to demonstrate that our approach offers better scalability compared to other state-of-the-art methods.

The rest of the paper is organized as follows. In Section 2, we describe previous proposals that address the problem of team formation in the classroom, highlighting the main contributions found in the literature. In Section 3, we present the formal description of the team formation problem. In Section 4, we detail our proposal. Section 5 presents the different experiments carried out to validate our proposal, comparing it with other relevant state-of-the-art proposals. Finally, Section 6 highlights the main conclusions of our proposal and future lines of work.

2. Team formation in the classroom

The problem of finding optimal teams has attracted the attention of the scientific community in recent years. Some authors have proposed algorithms to form teams in business and collaborative settings. For instance, [18] proposed an online algorithm to form teams as tasks arrive sequentially by minimizing communication overhead and ensuring a balanced workload. Later, Gutierrez et al. [19] proposed several metaheuristics and constraint programming approaches to form teams in organizations where multiple projects must be carried out simultaneously. The authors propose algorithms that can decide the dedication of different workers for the multiple projects that must be tackled. The authors in [20] also propose a mathematical optimization model to form a team of experts for a given task that, apart from maximizing cooperation among team members, also ensures that the team is reliable and robust to changes.

The team formation in the classroom problem [21,22], also known as group formation depending on the specific work, is a variant of the team formation problem that has some special considerations. Firstly, rather than forming a single team from a subset of candidates, the entire classroom must be partitioned into non-overlapping teams. Secondly, it is implicitly assumed that all of the teams should carry out similar tasks in terms of complexity and required skills since there is an educational goal behind the formation of the teams and the activities that they will carry out. Thirdly, teams should be similar in size. The reader is addressed to the surveys of Maqtary et al. [22] and Odo et al. [21] for a comprehensive field analysis. In the literature, several approaches for team formation in the classroom have been proposed, ranging from simple ones such as grouping students according to their average grade or the time they have available for work, to more complex approaches based on personality and behavior such as Belbin's role taxonomy [7,10,14,15], the Myers-Briggs Type Indicator [16,23,24], or the Big Five Inventory [17,25,26].

Some proposals have used some of these criteria for team formation in the classroom. Varvel et al. [23] applied Myers-Briggs as the criterion for forming teams in various university degrees. In [13], the criterion used is the students' online activity on a MOOC platform to group them according to similar activity levels and possibly similar learning aspirations. In [12], class sociograms and Belbin's taxonomy are utilized. Komlenov et al. [27] considered the demographics of the students as well as the number of Belbin roles present in each team. Ma et al. [25] proposed an integer linear programming model for forming teams that considers multiple characteristics, including the Big Five personality traits, to assess the value of a student as a leader. Moreover, we have used Belbin roles as perceived by other peers [4] and, more recently, we presented a comparison of Belbin's taxonomy with Myers-Briggs [15]. Although these works provide functionalities for team formation and have been applied in the classroom, they have considered a limited number of students, making it feasible to obtain the optimal solution in a reasonable time.

Finding an optimal partition of students into teams is a highly combinatorial problem that is difficult to solve manually and using algorithms. Many variants of the team formation problem are NP-hard problems [5]. In [6], we proposed an integer linear programming model for solving the team formation problem in the classroom, allowing for incorporating team heterogeneity characteristics. We tested our proposal with different solvers and several problem instances generated from a real dataset of students' traits regarding Belbin's role taxonomy and Myers-Briggs. One of the experiments' conclusions is that no solver was able to solve problem instances with a size of 60 and teams of 5 students. This indicated the need to study other models that scale better with larger instances.

The use of optimization techniques is necessary to tackle this problem appropriately. In relation to this issue, numerous papers in the literature have proposed the use of heuristic and metaheuristic algorithms to tackle the problem of team formation in the classroom. Yannibelli et al. [10] proposed a genetic algorithm to group students using an objective function that considers a single criterion: the heterogeneity of team members according to Belbin's role taxonomy. The representation of the solutions is based on a list of students where each position indicates the student and each value, the team to which they belong. In the experiments, the authors used instances of up to 3000 students divided into teams of 6, which indicates that these models scale well to numbers that would not typically be used in a real classroom. In [9], a genetic algorithm for team formation using the Big Five Inventory as a criterion is proposed. Although this case was applied with real students, the maximum class size was less than 50 students.

Although using a single criterion for team formation is widely extended, more recent proposals focus on incorporating more than one criterion in the objective function. For example, Andrejczuk et al. [11] propose an algorithm that uses personality criteria based on Myers-Briggs, as well as incorporating synergy between students and the level of competence of students to carry out different tasks. The algorithm used is based on local search, and they compared their results with an integer linear programming solution. Although the total number of students to be grouped was not high, the solutions obtained by the algorithm were very good compared to the optimal solution. Garshasbi et al. [28] also proposed a multi-objective genetic algorithm to foster

homogeneity across groups and heterogeneity within groups. The proposed generic model would allow the integration of characteristics such as gender or different skills. Still, it was evaluated with synthetic data and was not based on real data.

Ugarte et al. [7] proposed a genetic algorithm to form groups using a multi-objective function that combines Belbin roles with gender. In this case, the paper's novelty is that it defines a multi-objective function that incorporates information from the entire Belbin role vector rather than information from the predominant roles. Moreno et al. [8] also proposed a genetic algorithm to solve the team formation problem. In this case, different characteristics of the students were incorporated, which may be related to gender, academic grades, or personality traits, among others. Unlike similar proposals, the authors use a matrix-based representation where the rows represent the groups to be formed, and the columns the size of the groups. This representation allows for applying a genetic operator proposed by the authors, where different crossover points are chosen for each row.

As can be seen, these proposals address the team formation problem using several metaheuristic techniques that allow for the incorporation of different criteria. However, these proposals have two significant limitations when forming teams. On the one hand, all of them assume the teams have the same size or that the lecturer specifies the number of desired teams, and the algorithms calculate the size for each of the teams, possibly with a few teams having an extra member. On the other hand, to our knowledge, no previous proposal explicitly considers combinations of students who must work together or those who must not. Although in real practice, lecturers use these restrictions [15], the objective functions employed in the literature do not consider these aspects that capture human relationships in the classroom.

3. Problem description

This section describes and formalizes the team formation in the classroom problem. Let us assume we have a classroom formed by $\mathcal{S} = \{s_1, \dots, s_n\}$ students. We define a team $T \subset \mathcal{S}$ as a subset of the students in the classroom, and $\mathcal{L} = \{l_1, \dots, l_r\}$ represents a set of valid team sizes for the activity that needs to be undertaken by the teams. Each valid size $l \in \mathcal{L}$ has a minimum and a maximum number of teams that may have such size, expressed as m_l and M_l.

In addition to this, we denote by $\mathcal{P}(\mathcal{S})$ the power set of the students, that is, the set of all of the subsets of students. $\mathcal{C} \subset \mathcal{P}(\mathcal{S})$ represents subsets of students that should be placed together into the same team. For instance, if $C \in \mathcal{C}$, then that means that every student in C has to be placed into the same team. We will refer to $\mathcal{C}$ as the set of all the compulsory combinations of students and any $C \in \mathcal{C}$ as a compulsory combination of students. Similarly, we define by $\mathcal{N} \subset \mathcal{P}(\mathcal{S})$ subsets of students that should never be placed into the same team. For instance, if $N \in \mathcal{N}$ then $\forall s_i, s_k \in N$, s_i, and s_k should never be placed into the same team. We will refer to $\mathcal{N}$ as the set of all forbidden combinations of students and any $N \in \mathcal{N}$ as a forbidden combination of students. Given these definitions, we say that a team T is feasible if and only if it complies with the three following conditions:

1. $|T| \in \mathcal{L}$
2. $\forall C \in \mathcal{C},\ T \cap C = \emptyset \ \vee\ T \cap C = C$
3. $\forall N \in \mathcal{N},\ |T \cap N| \leq 1$

In other words, we say that a team T is feasible if and only if its size is one of the valid team sizes, it contains all of the students from any compulsory combination of students or no students from that combination, and it includes at most one student from any forbidden combination of students. For the sake of simplicity, we define by $\mathcal{T}$ the set of all feasible teams.

A team partition $P = \{T_1, \dots, T_m\}$ is a partition of all students into non-overlapping teams that include all the students in the classroom. That is, $\bigcup_{j=1}^{m} T_j = \mathcal{S}$ and $\forall T_j, T_k \in P,\ T_j \cap T_k = \emptyset$. We say that a team partition P is feasible if and only if $\forall T \in P$, then $T \in \mathcal{T}$ and the number of teams with a size l is between m_l and M_l. For simplicity, let us denote the set of all feasible team partitions by $\mathcal{P}$. The quality of a feasible partition P of students into teams is measured by the function $\mathcal{F}(P)$. With these definitions, the team formation in the classroom problem consists of finding the optimal team partition in $\mathcal{P}$ according to function $\mathcal{F}(.)$. Different criteria have been employed in the literature to measure the quality of dividing students into teams. For instance, based on personality traits, based on the skills of team members, gender, or academic criteria [4,8,11,29]. Generally, $\mathcal{F}(P)$ can be calculated by combining (in any way) the value provided by each of the teams in the partition. The evaluation of an individual team in the partition is represented by the function $f(T)$. According to previous definitions, one can formalize the team formation in the classroom problem using a standard form:

$$\max \quad \mathcal{F}(\delta_1, \dots, \delta_{|\mathcal{T}|}) \tag{1a}$$

s.t.

$$\sum_{t_i \in \mathcal{T}} \alpha_{i,j} \cdot \delta_i = 1 \qquad \forall s_j \in \mathcal{S}, \tag{1b}$$

$$\sum_{t_i \in \mathcal{T}} \lambda_{i,l} \cdot \delta_i \geq m_l \qquad \forall l \in \mathcal{L}, \tag{1c}$$

$$\sum_{t_i \in \mathcal{T}} \lambda_{i,l} \cdot \delta_i \leq M_l \qquad \forall l \in \mathcal{L} \tag{1d}$$

where δ_i is a binary decision variable that describes whether or not feasible team $T_i \in \mathcal{T}$ is included in the team partition, $\mathcal{F}$ is the function that evaluates the selected team partition, $\alpha_{i,j}$ is a constant that is equal to 1 when feasible team T_i includes student s_j and 0 otherwise, and λ_{i_l} is a constant that is equal to 1 when feasible team T_i has l members and 0 otherwise. Finally, constraint (1b) guarantees that each student is placed into exactly one team, and constraints (1c) and (1d) guarantee that the number of teams for each size is between their corresponding minimum and maximum number. As one may observe, the model assumes that all feasible teams $T_i \in \mathcal{T}$ are pre-computed for the construction of the model. There is one decision variable per feasible team and as many constraints as the number of students and twice the number of valid team sizes. When the function $\mathcal{F}(.)$ can be expressed as the individual contribution of each team, that is $\mathcal{F}(P) = \sum_{T_i \in \mathcal{T}} f(T_i) \cdot \delta_i$, one can employ integer linear programming solvers to provide the optimal solution for the previous optimization problem. For non-linear terms in $\mathcal{F}$, one may need to resort to non-linear integer programming solvers.

The optimization problem has differences and similarities with other optimization problems solved in the literature. Constraint (1b) is inherent to team formation in classroom problems, and all existing proposals support this feature. However, there is mixed support for the rest of the constraints. As stated above, previous works do not allow for specifying variable team sizes. Constraints (1c) and (1d) provide a more flexible way to specify the structure of the desired team partition. For instance, one can configure solutions where only one team size is allowed by specifying a valid team size and making $m_l = M_l$. It is also possible to provide a desired number of teams and calculate how many teams of each size are required, again $m_l = M_l$. In addition to all of this, the user can define team sizes that would be appropriate for the task at hand, specify a minimum and maximum number of teams per size (even 0 or infinite, respectively), and let the algorithm decide what would be the best structure for the team partition according to the objective function. Hence, it provides more flexibility in this regard. Finally, although it is not explicit in the model mentioned above as it works directly with feasible teams, the model also considers explicitly combinations of compulsory and forbidden students. As stated above, this feature is missing in other state-of-the-art algorithms but is important in practice.

While the optimization problem described in this section is flexible and can be used to model various situations and scenarios faced in the

classroom, it has several limitations. First of all, it has been shown in the literature that when using exact methods for similar optimization problems (even in the linear objective case), these solutions have scalability problems when the size of the classroom increases or when different team sizes are considered [6]. Second, calculating the optimal solution using an exact method may be prohibitive for applications requiring a near real-time response for the user [6]. In addition, exact methods often require precise problem formulations and complete data, which can be impractical in real-world educational settings where data may be incomplete or uncertain. Metaheuristics, in contrast, are more robust to such imperfections and can provide good solutions even with noisy or partial data.

4. Proposal

In this section, we describe the details of our proposed metaheuristic. Evolutionary algorithms inspire our metaheuristic since it is based on a population of solutions, genetic operators and selection. However, unlike classic genetic algorithms, the only genetic operator employed in this case is a mutation rather than the combination of crossover and mutation. In this sense, the proposed metaheuristic algorithm is similar to evolution strategies, as many variants of this metaheuristic tend to employ only mutation [30,31]. In addition, our algorithm also incorporates local search for some of the solutions in the population, which introduces a component common in memetic algorithms [32].

First, we describe how the fitness function is handled to deal with the problem's constraints. Then, we describe the concept of synergy between team members, which we employ to drive several decisions in our proposal. After that, we describe team partition structures and how they are employed to describe solutions in our proposal. Then, we describe the classic evolutionary components of population initialization, the mutation operators, the selection operator, population replacement, and the local search procedure. Finally, we present an overview of the metaheuristic algorithm.

4.1. Fitness function and constraint handling

The proposed metaheuristic's fitness function depends on the problem to be solved. As the metaheuristic aims to be general, it could take any objective function $\mathcal{F}(P)$ that is calculated as the combination (in any form) of the individual values provided by each of the teams. However, we modified this function to handle the problem's constraints. While many of the decisions made in the algorithm aim to minimize the violation of constraints of the problem, it is still possible to generate unfeasible solutions as the process of the different algorithmic decisions in the search. Therefore, we must include an extra mechanism in our metaheuristic for unfeasible solutions. In our case, we decided to penalize the value provided by an invalid team by taking away from its value a large negative number whose value depends on how close the team is to being valid. This way, our metaheuristic will prefer feasible to unfeasible solutions when operating. Particularly, the modified fitness function $\mathcal{F}'(P)$ can be expressed as:

$$\begin{aligned}\mathcal{F}'(P) = -\mathcal{M}\cdot\sum_{N\in\mathcal{N}}\sum_{T\in P}\frac{max(|T\cap N|-1,0)}{|N|}\\ -\mathcal{M}\cdot\sum_{C\in\mathcal{C}}\sum_{T\in P}\frac{|C-T|\cdot\lambda_{T,C}}{|C|-1}\\ +\mathcal{F}(P)\end{aligned} \tag{2}$$

where $\mathcal{M}$ is a large constant and $\lambda_{T,C}$ is a binary constant that indicates with a 1 if the team T and the compulsory combination C have any elements in common, 0 otherwise. The first term in the sum represents how far the solution is from complying with the forbidden combinations constraints, the second term represents how far the solution is from complying with compulsory combinations constraints, and the third term is the original objective function of the problem. In our algorithm, there is no need to penalize invalid solutions due to invalid team sizes since our proposal does not generate such solutions.

4.2. Synergy estimation

In some situations, the whole fitness function $\mathcal{F}(P)$ may be expensive to compute, it may be a black-box function, or we may be interested in pre-evaluating local decisions such as swaps between teams, or the impact of adding a new member to a team without entirely evaluating $\mathcal{F}(P)$ or $f(T)$. That is, sometimes, we need a quick and inexpensive approximate function to assess the quality of the decisions that we make during the search process. Ideally, as our metaheuristic aims to be general and applicable to several team formation problems in the classroom, this approximate function should be based on general aspects common to many team formation problems rather than problems specific to one objective function.

One of the assumptions made by our metaheuristic is that a team is as good as the synergies between their team members. Therefore, the assumption is that the appropriateness and quality of a team are related to the synergies between its specific team members. This is not an unrealistic assumption, as it is well-known that team members ideally should complement each other and be a good match [33]. This idea is general and independent of the underlying objective function employed to evaluate a team partition and, hence, prospectively applicable to many scenarios.

Let us denote by $\psi(s_i, s_j)$ the synergy between student s_i and student s_j. We assume that the overall quality of a team $T = \{s_1, \dots, s_k\}$ can be approximated as function of the synergies between its team members $f(T) \approx f'(\psi(s_1, s_2), \dots, \psi(s_1, s_k), \dots, \psi(s_{k-1}, s_k))$. In practice, one may only have access to an estimation or educated guess on this synergy metric $\hat{\psi}(s_i, s_j)$. Different ways exist to estimate this synergy $\hat{\psi}(s_i, s_j)$. For instance, one may employ a team evaluation function $f(T)$, if it exists, and assume that the synergy between two team members is approximately the quality of a team only composed of students s_i and s_j. As another example, one may design a specific function to approximate $\hat{\psi}(s_i, s_j)$ based on some well-known criteria, similarly to how the objective function $\mathcal{F}(P)$ is designed. Finally, one could approximate $\hat{\psi}(s_i, s_j)$ by estimating based on evaluations of the objective function $\mathcal{F}(P)$ in solutions where s_i and s_j are placed in the same team. In this article, we have implemented the first option $\hat{\psi}(s_i, s_k) = f(\{s_i, s_k\})$, although it is acknowledged that other approximations are plausible.

In our metaheuristic, we employ the synergy estimation $\hat{\psi}(s_i, s_j)$ in several algorithmic decisions. Typically, we employ the synergy between two members to estimate the potential synergy between a student or a set of students and an already existing set of students. We employ $\hat{\Psi}(S_1, S_2)$ to refer to the synergy between students in set S_1 and students in set S_2. In our current implementation, we approximate this synergy between both sets of students as:

$$\hat{\Psi}(S_1, S_2) = \frac{1}{|S_1|\cdot|S_2|}\sum_{s_j\in S_1}\sum_{s_k\in S_2}\hat{\psi}(s_j, s_k) \tag{3}$$

that is the average synergy between members from both sets of students.

4.3. Chromosome representation

As mentioned, a feasible team partition $P = \{T_1, \dots, T_m\}$ is a partition of all of the students into non-overlapping feasible teams that include every student in the classroom and where the total number of teams with a given size is between a minimum and maximum. We define the team partition structure of P as $\pi = (|T_1|, \dots, |T_m|)$, thus being the cardinality of the teams that compose the team partition. For a given classroom with $|S|$ students, many feasible team partition structures may result in different team compositions. For instance, for a class of 12 students with 2 and 3 being valid team sizes, feasible team partition structures would be $[3, 3, 3, 3]$, $[3, 3, 2, 2, 2]$ and $[2, 2, 2, 2, 2, 2]$. It should be highlighted that a team partition structure only defines how many teams will be employed in the solution and the sizes of each of

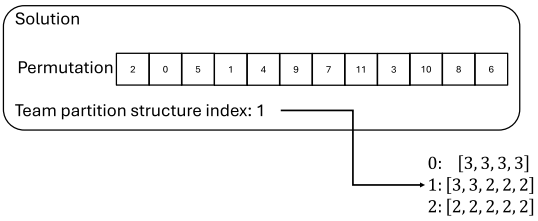


Fig. 1. Example of the representation of a solution.

the teams, and it does not define what specific students form each of the teams.

Since the optimization problem posed in Section 3 allows for different team sizes and even a minimum and a maximum number of teams with a given size in the final solution, defining an appropriate team partition structure is important. Our proposed metaheuristic relies on valid team partition structures to represent a solution. Hence, at the initialization of the evolutionary algorithm, our proposed method calculates all of the valid team partition structures Π for the problem.

The proposed evolutionary algorithm employs two elements to represent a solution to the problem of team formation in the classroom. The first element is an array in the form of a permutation of all student identifiers (from 0 to $n-1$). This vector would suffice to represent a single solution if only one team partition structure was possible. The second element is the specific index of the employed team partition structure, according to the valid team partition structures generated at the start of the algorithm.

Fig. 1 shows an example of how a solution is represented in our algorithm for the basic scenario employed in the previous subsection (i.e., 12 students, unlimited number of teams of 2 and 3 students). The team partition structure index defines that students will be divided into 2 teams of 3 members and 3 teams of 2 students. The specific students that will be placed on each team are defined by the permutation of the students. Therefore, in the solution represented in the figure, the teams in the solution are $[2, 0, 5]$, $[1, 4, 9]$, $[7, 11]$, $[3, 10]$, and $[8, 6]$.

4.4. Population initialization

Our proposed metaheuristic is population-based and, thus, initially, it needs a set of different solutions to work on. Similar to other works [34,35], we found that using a heuristic to generate the initial pool of solutions tends to provide better solutions than employing random initialization. In our case, we have designed a randomized greedy heuristic that can generate solutions that generally avoid violating the problem's constraints. The pseudo-algorithm of this randomized greedy heuristic can be found in Algorithm 1. The heuristic first selects a random team partition structure π from the list of all valid team partition structures Π, and an empty team partition P is initialized using as many teams as the teams specified in π and with the sizes also specified by π. Then, the heuristic initializes a set with all students that still need to be placed in the solution S'.

Lines 5–11 contain the logic to allocate as many compulsory combinations of students $C \in \mathcal{C}$ as possible. It works by iterating over all of the compulsory combinations of students and then randomly selecting a team that has enough space for all of the students in the compulsory combination of students. If such a team is found, the students are added to the team, and S' is updated.

Then, lines 12–16 aim to populate currently empty teams. It does so by randomly selecting a student who has not been assigned to a team and adding him/her to that team. After these steps, we have every team assigned at least one student, and possibly, many students within the same compulsory combination have been placed together.

Finally, lines 17–22 are in charge of making the randomized greedy decisions. This part of the heuristic works by randomly iterating over

Algorithm 1 Generation of a solution for the initial population

```
1: procedure RANDOMIZED GREEDY HEURISTIC
2:     select random team partition structure π ∈ Π
3:     initialize empty teams for each T ∈ P
4:     S' ← S
5:     for each compulsory combination C ∈ 𝒞 do
6:         select random T ∈ P with |T ∪ C| ≤ π(i)
7:         if T exists then
8:             T ← T ∪ C
9:             S' ← S' − C
10:        end if
11:    end for
12:    for each empty T ∈ P do
13:        select random s_j ∈ S
14:        T ← T ∪ {s_j}
15:        S' ← S' − {s_j}
16:    end for
17:    for each s_j ∈ S' do
18:        roulette wheel valid team T, according to Ψ̂(T, {s_j})
19:        T ← T ∪ {s_j}
20:        S' ← S' − {s_j}
21:    end for
22:    return P, π
23: end procedure
```

students that still need to be assigned. For each student s_j, the heuristic analyzes each team T that still has a place for the student. It calculates the average synergy of team members in that team with the selected student $\hat{\Psi}(T, s_j)$. In this step, we penalize students within the same forbidden combination $N \in \mathcal{N}$ by penalizing the estimated synergy between those students. This way, our heuristic may avoid decisions where students within the same forbidden combination are put into the same team. After analyzing every option, a team is selected with a probability proportional to $\hat{\Psi}(T, s_j)$ using roulette wheel selection. Once all of the students have been allocated, the algorithm finishes and returns the resulting team partition P and the associated team partition structure.

4.5. Mutation operators

As mentioned, our evolutionary method is strongly guided by the mutation operator. In some preliminary experiments, while we were designing our algorithm, we realized that the algorithm's performance in terms of fitness improved more with a higher emphasis on mutation operators than with classic crossover operators on permutation like order crossover, edge crossover, or position crossover [36]. The preliminary experiments suggested that saving the time spent in crossover and using mutation as much as possible was better. Hence, we designed our proposed metaheuristic with a high emphasis on mutation.

Two elements can be mutated in our chromosome representation: the team partition structure and the students' permutation. Consequently, we designed two different mutation operators, each focusing on mutating a part of the chromosome. Both operators are activated when certain conditions are met, which will be described when we present the general outline of our metaheuristic (see Section 4.8). Next, we describe each of the mutation operators in detail.

4.5.1. Team partition structure (TPS) mutation

In some situations, as highlighted in Section 4.3, it may be possible to have classroom partitions with different numbers of teams and different team sizes. Since it is not possible to know beforehand which team partition structure one may find the optimal solution according to the objective function, it is necessary to have operations that allow one to explore other team partition structures. We have designed a special mutation operator that takes this role.

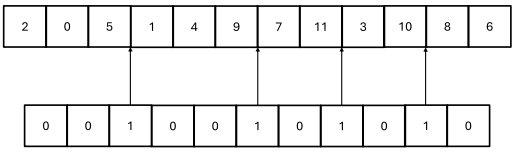


Fig. 2. Example of the representation of a team partition structure as a binary vector.

The algorithm works by mutating the current team partition structure to a similar one, thus avoiding drastic changes in the team composition. Therefore, the mutation relies on the ability to measure the similarity between two team partition structures. For the mutation operation, each team partition structure is represented by a binary vector representation. This translation between team partition structure and binary representation is done only once at the start of the proposed algorithm. The binary vector representation is used to calculate similarities between team partition structures, as defined in the work of [37] for coalition structures. If there are n students, the binary vector representation of the team partition structure consists of binary vector with $n-1$ positions. Each position in the binary vector is associated with a prospective cut point in the permutation vector. If the permutation chromosome has n positions, there are $n-1$ possible cut points. A 1 in the ith position of the binary vector indicates that the ith element in the permutation chromosome acts as the last member of a team. The binary vector is to be processed sequentially to be interpreted. Initially, the first team will be composed of the corresponding students in the permutation vector between the first position and the position marked by the first appearance of a 1 in the binary vector (included). Then, the next team will be composed by corresponding students from the next position to the next appearance of a 1 in the binary vector (included), and so forth. Fig. 2 shows how this representation should be interpreted. Let us say that the team partition structure is [3, 3, 2, 2, 2]. Its representation as a binary vector would be [0, 0, 1, 0, 0, 1, 0, 1, 0, 1, 0]. The first team includes all students in the permutation up to the first one (third position). Therefore, the first team is composed of [2, 0, 5]. The next team will include members from the fourth position in the permutation up to the position marked by the next 1 in the binary vector. In this case, the sixth position. Thus, members [1, 4, 9] will form the next team. Then, the next team will include members in the permutation from the seventh member to the position marked by the next 1 in the binary vector. So, from the seventh to the eighth member (i.e., [7, 11]). Similarly, the next team will include from the ninth member in the permutation to the tenth member (i.e., [3, 10]). Finally, it will include from the eleventh member in the permutation up to the last position of the vector as there are no more 1's (i.e., [8, 6]).

It should be noted that this representation is solely employed for computing the similarity between team partition structures; internally, each solution only has the index of the assigned team partition structure. As mentioned, this conversion between team partition structure (index) and binary vector representation is carried out only once at the start of the proposed algorithm. Once all team partition structures have been generated, we calculate the corresponding binary vector representation with a simple algorithm. We initialize an index variable to zero. For each element in the team partition structure, we update the index variable by adding the element and setting the position indicated by the index variable in the binary vector to 1.

Then, we employ the Levenshtein distance with the binary vector representation to measure the similarity between pairs of team partition structures:

$$Sim(\pi, \pi') = 1 - \frac{Levenshtein(\pi, \pi')}{|S|} \tag{4}$$

where π and π' are two team partition structures and $|S|$ is the number of students in the classroom. The calculation of all valid team partition structures and the similarities between pairs of valid team partition structures are precomputed at the start of our proposed method.

Suppose the metaheuristic decides to mutate a team partition structure π. In that case, it will do so by selecting a new π' using the similarity between the team partition structures as expressed above, sorting them, and then randomly selecting using exponential ranking selection [38] with probability:

$$p_i = \frac{c-1}{c^{|\Pi|-1}-1} c^{|\Pi|-i-2} \tag{5}$$

where Π represents all of the valid team partition structures, i is the ranked position of the team partition structure π' according to the similarity (ascending, from 0 to $|\Pi|-2$), and c is a hyperparameter that controls the exponential behavior of the selection procedure, with values closer to 1 behaving more like a uniform selection and values closer to 0 behaving more exponentially.

4.5.2. Permutation mutation

This mutation is in charge of changing the composition of the teams by introducing a small perturbation to the current team partition. The mutation operator is based on the classic swap mutation [39] operator for permutations. Still, it introduces some variations to adapt it to the current problem and to boost its performance in practice. More specifically, the mutation operator may swap multiple team members between two teams. This behavior is important when considering compulsory combinations of students, as swapping just one member may consistently violate some of the problem's constraints. Moreover, the proposed mutation operator also explores and optimizes the current solution's local neighborhood. The details of the mutation operator can be found on Algorithm 2.

Algorithm 2 Mutation operator for mutating permutations of students

```
1:  procedure PERMUTATION MUTATION(P, n_mut)
2:      randomly select T_i, T_k ∈ P
3:      d ← min(|T_i|, |T_k|)
4:      Initialize W to zeros
5:      for n_mut repetitions do
6:          m ← randint(1, d)
7:          S'_1, S_1 ← random split of T_i into (m, |T_i| − m) students
8:          S'_2, S_2 ← random split of T_k into (m, |T_k| − m) students
9:          W(S_1, S_2) ← Ψ(S_1, S'_2) + Ψ(S_2, S'_1)
10:     end for
11:     roulette wheel swap of S_1 ∈ T_i, S_2 ∈ T_k according to W
12:     swap S_1 and S_2 between T_i and T_k in P
13:     return P
14: end procedure
```

The mutation operator works by mutating a single solution. More specifically, it modifies the team composition by swapping members between two teams selected randomly. Lines 2–4 randomly select two teams T_i and T_k that will swap members. Then, the mutation operator calculates the maximum number of members that can be swapped between both members. After that, the mutation operator simulates the impact of n_{mut} random swap operations between T_i and T_k (lines 5–10). For each operation, it selects a random number of members to be swapped m and then randomly splits each team into two sets, one with m members and the other with the rest of the team members. Then, the average synergy between members of the current team and the members to be received in the swap operation is calculated for both teams. The total synergy arising from the proposed swap operation is the sum of both synergies, which is stored for later selection of the specific swap operation to be performed. After simulating the different swap operations, the mutation algorithm uses roulette wheel selection (i.e., fitness proportionate selection) to select the final mutation operation (lines 11–13).

4.6. Selection operators and population replacement

In this evolutionary algorithm, the selection operator decides which individuals are selected to undergo mutation operations. We employ tournament selection [40] for selecting $\alpha\%$ of the individuals of the population without replacement. When carrying out a tournament, $\beta\%$ of the individuals in the population are sampled. Then, the fittest individual is selected with a probability of p_t, and another individual is selected otherwise. The procedure is repeated until $\alpha\%$ of the individuals in the population have been selected.

After children have been generated with the mutation operations, deciding what individuals are kept in the population for the next generation is necessary. In this evolutionary algorithm, we employ probabilistic crowding [41]. This population replacement strategy aims to keep diversity in the population by introducing competition among similar individuals. In our case, it works by making parents and offspring generated from those parents compete with each other. Since our evolutionary algorithm is based on mutation, a parent competes with its mutated child. The fitness of the individuals regulates the competition, as the fittest solution among both is kept for the next generation with a probability of p_{pc}. The other solution is selected otherwise. The p_{pc} hyperparameter controls how elitist the metaheuristic should be or whether or not it should allow more exploration. Typically, this probability is high but leaves room for solutions with worse fitness to be kept in the population with the hope that more exploration will yield better results.

4.7. Local search

Our proposed algorithm also employs local search to improve some solutions in the population. The proposed local search works sequentially, carrying out several local improvement steps for a solution. A local improvement step consists of trying to improve the current solution by doing a swap operation between two teams. The local improvement step follows a first improving strategy [42]. In essence, the local improvement steps whenever an improvement has been found. The procedure details for a local improvement step can be found in Algorithm 3.

Algorithm 3 Local improvement procedure

```
1: procedure LOCAL IMPROVEMENT STEP(P, π)
2:     F ← F'(P)
3:     for each team T_i ∈ P in random order do
4:         for each team T_k ∈ P in random order do
5:             for swap s ∈ T_i, s' ∈ T_k in random order do
6:                 T'_i ← T_i − {s}
7:                 T'_k ← T_k − {s'}
8:                 ΔF'(T'_i) ← Ψ̂(T'_i, s') − Ψ̂(T'_i, s)
9:                 ΔF'(T'_k) ← Ψ̂(T'_k, s) − Ψ̂(T_k, s')
10:                if ΔF'(T'_i) + ΔF'(T'_k) > 0 then
11:                    swap s and s' in P
12:                    if F'(P) > F then
13:                        return P
14:                    else
15:                        undo swap between s and s' in P
16:                    end if
17:                end if
18:            end for
19:        end for
20:    end for
21:    return P
22: end procedure
```

The local improvement step works by iterating over all of the current teams $T_i \in P$ in random order and then iterating for the rest of the teams $T_k \in P$ on every possible swap between students in both teams $s \in T_i, s' \in T_k$ (lines 3–5). The local improvement step then calculates the impact on each team's synergy by measuring the synergy gained by the member transferred, and the member received (lines 6–9). If globally, a better synergy is achieved, then the swap operation is carried out, and the fitness of the new solution is computed. If the current solution has improved, the local improvement step finishes. Otherwise, the swap operation is undone, and the local improvement step proceeds by analyzing the rest of the prospective swaps. This process is documented in lines 10 to 16. Please note that the synergy estimation is first evaluated before the actual fitness function, as the fitness function requires checking if the solution is valid and is more costly to evaluate than the synergy estimation. The actual swap and the fitness function are computed only if there is an indication that the solution may improve.

The local improvement step procedure may be applied several times to the same solution. It can be applied a maximum of n_{ls} times on the current solution during the local search. If no improvement can be made to the solution in a local improvement step, then the local search ends for that solution.

4.8. Algorithm overview

After describing all of the main components of our metaheuristic proposal, we describe the general flow of our proposed metaheuristic in this subsection. The specific details of the algorithm can be found in Algorithm 4.

Algorithm 4 Proposed metaheuristic overview

```
1: procedure TEAM FORMATION METAHEURISTIC
2:     Calculate all valid team partition structures Π
3:     Calculate the similarity between every π, π' ∈ Π
4:     Create pop_size solutions with RandomizedGreedyHeuristic
5:     while not stop condition do
6:         Parents ← TournamentSelection(α, β, p_t)
7:         for each pair of parents P_1, P_2 do
8:             P'_1 ← P_1, P'_2 ← P_2
9:             π_best ← π_1 if F'(P_1) > F'(P_2) else π_2
10:            Mutate(P'_1)
11:            Mutate(P'_2)
12:            Change π_1 to π_best with probability p_b else MutateTPS(π_1) with probability p_mt
13:            Change π_2 to π_best with probability p_b else MutateTPS(π_2) with probability p_mt
14:            ProbabilisticCrowding(P_1, P'_1, p_pc)
15:            ProbabilisticCrowding(P_2, P'_2, p_pc)
16:        end for
17:        Roulette wheel P from best γ% in population
18:        LocalSearch(P) with probability p_ls
19:    end while
20: end procedure
```

First, before entering the evolutionary loop, the algorithm generates all valid team partition structures $\pi \in \Pi$ since they are needed to generate initial solutions. Then, the algorithm calculates the similarity between each pair of team partition structures $\pi, \pi' \in \Pi$ using the procedure described in Eq. (4). After that, the algorithm employs the Randomized Greedy Heuristic described in Algorithm 1 to generate a total of pop_{size} solutions that constitute the initial population of solutions. These three steps are described in lines 2 to 4.

Then, the algorithm enters into the evolutionary loop. In the pseudocode, it is specified that the evolutionary loop continues while the stop condition is not met. This stop condition is general, and it could be based on computation time, a maximum number of generations, a maximum number of calls to the fitness function, or even a combination of those. As it will be described in the experiments section, we employed a maximum computation time for all of the methods. Still, it could be interchanged by other stop criteria with ease.

In the evolutionary loop, the first step consists of selecting with tournament selection, as described in Section 4.6, the solutions that will be evolved with the mutation operators. Then, pairs of non-overlapping parents are considered (line 7). The team partition structure of the best

Table 1
List of hyperparameters of our metaheuristic and their description.

Population initialization	
pop_{size}	Size of the population
Team partition structure mutation	
p_b	Probability of inheriting the fittest team partition structure
p_{mt}	Probability of mutating the team partition structure
c	Exponential behavior for the ranking selection
Permutation mutation	
n_{mut}	Number of swap operations to simulate
Selection	
α	Percentage of the population to be selected for evolutionary operations
β	Percentage of the population to be used for each tournament
p_t	Probability of selecting the winner of the tournament
Population replacement	
p_{pc}	Probability of keeping the fittest individual between parent and child
Local search	
γ	Percentage of the fittest individuals to use for roulette wheel selection
p_{ls}	Probability of applying local search at the end of each iteration
n_{ls}	Maximum local improvement steps to apply in each local search

of the two parents, π_{best}, is obtained (line 9). Then, children P_1' and P_2' are generated by mutating both parents.

Then, solutions may change their team partition structure and, consequently, its team member composition (lines 12–13). First, the team partition structure of the fittest from the two parents is inherited with a probability of p_b. This inheritance mechanism adds exploitation and elitism to the search carried out in the team partition structure. This way, gradually, the population may converge toward team partition structures that are more appropriate to form teams according to the objective function. If inheritance is not activated, the team partition structure will mutate with a p_{mt} probability. The mutation carried out is the one based on the similarity between team partition structures and described in Section 4.5.1.

Once the mutations have been carried out, we apply probabilistic crowding with parents and children (lines 14–15) to decide what individuals are kept in the population. Finally, once all of the pairs of parents have been processed, the best γ% of the solutions in the population participate in a roulette wheel selection (line 17). This roulette wheel process selects the solution that, with a probability of p_{ls}, will undergo the local search process defined in Section 4.7. To conclude, Table 1 briefly describes all of the hyperparameters employed by our metaheuristic.

5. Experiments

This section describes the experiments we carried out with our proposed algorithm. The main goals of these experiments are: (i) to show that, despite being designed for the specific case of problems with constraints, our method achieves a good performance for classic instances that involve no special constraints; (ii) to show that our method achieves good performance for scenarios that involve constraints related to the valid team sizes, and the existence of compulsory and forbidden combinations of students; (iii) to show that our proposed method's performance is robust to the type of objective function employed to evaluate the quality of the team partition; and (iv) to show that our proposed method scales better with the problem size than other alternatives in the literature.

We start this section by describing the different problem instances used in the experiments. Next, we describe the state-of-the-art algorithms that have been implemented to compare our algorithm with. Then, describe and analyze the experiments that we have carried out in scenarios with no special constraints and in scenarios with constraints.

5.1. Problem instances

Throughout the years, scholars working on team formation in classroom problems have proposed different objective functions to evaluate both the whole team partition $\mathcal{F}(P)$ and individually each team $f(T)$. As we aim to have a robust proposal to the type of objective function (objective iii), we will run and compare our method in various problem instances. Ideally, we should test our algorithm with realistic problem instances to correctly assess the performance of our proposal in a real setting. However, it should be noted that many datasets employed to define problem instances in team formation in classroom problems are not publicly available for free use. Therefore, this fact limits the choice of objective functions that we can employ in our experiments. From our previous studies [15], we have collected an anonymous dataset of 260 student profiles that completed the Belbin self-perception inventory [43] and the Myers-Briggs personality test [44] in the Bachelor Degree in Tourism at Universitat Politècnica de València. The formation of heterogeneous teams concerning these personal traits are two popular criteria employed in the literature [7,10–12,15,45]. Belbin's theory identifies eight key behavioral roles that contribute to successful team dynamics. Each team member should ideally take on one or more of these roles to ensure effective teamwork. The foundations of Myers-Briggs focus on categorizing personality traits into four dimensions, each with two opposite preferences. Combining these preferences results in 16 distinct personality types, which can be used to form balanced teams based on complementary personality traits.

Hence, we mainly focused on solving problem instances whose objective functions are based on these two criteria, as we have a dataset that allows us to build realistic problem instances. Next, we describe each of the types of problem instances that we employed in our experiments.

- **Belbin role heterogeneity (BRH):** This objective function [15] works by separately evaluating each team. A team's score depends on the number of Belbin roles that members can play, with the maximum score being attained when all of the roles can be played. Then, the overall score of a team partition is the average of the teams' scores. The details for each formula can be found in their respective articles and will not be discussed in detail.
- **Belbin role heterogeneity with negative synergies (BRHNS):** The objective function is proposed in [10], and it works similarly to the previous one. However, some penalization is included when evaluating a team's score. If multiple team members can play the same role, the team's score is penalized to account for the fact that this could create conflict within the team. The team's score is penalized if no member can play some of the roles in Belbin's taxonomy.
- **Belbin role and gender heterogeneity (BRGH):** This objective function, proposed in [7,29], mixes two different criteria, Belbin's role taxonomy and gender heterogeneity. Each team is assessed based on both criteria. On the one hand, one aims to build a team whose average scores on each Belbin role are as high as possible. On the other hand, teams where we have the same number of male and female students are preferred. The global score of the team partition is calculated as the average score of the teams divided by the standard deviation of the team scores, thus aiming to make good and homogeneous teams in the score.
- **Myers-Briggs Heterogeneity (MBH):** This objective function [15] is based on the Myers-Briggs personality types. The assumption is that teams should have at least one member with each personality trait in the taxonomy. A team achieves its maximum score when, for each personality trait, at least two members have that trait. Then, a team partition score is calculated as the average of the teams' scores.

- **Homogeneous Team Characteristics (HTC):** This objective function, proposed in [8] and also later employed in [9], is based on the fact that a student can be characterized by a vector of traits (e.g., personality traits, skills, etc.). A team is then characterized as the deviation of the average of the team members' vectors to the average of the entire classroom. The team partition score is the sum of this deviation for each team in the classroom.

For each problem type described above, we generated realistic problem instances by sampling from our dataset of student records. In the case of HTC, as they are not explicitly based on Myer-Briggs or Belbin's role taxonomy, we generated students' traits by sampling each skill from a uniform distribution from 0 to 1. In this case, we generated problem instances with 4 and 8 skills (HTC4 and HTC8, respectively).

More specifically, first, we generated problem instances resembling the *classic problem instances* reported in the literature. We generated problem instances where no compulsory and forbidden student combinations exist, and each team should have the same size (i.e., 5 students in this case). We will refer to these instances as classic problem instances. We generated 50 problem instances for each combination of problem types and 40, 60, and 80 student classroom sizes. Hence, we generated $6 \cdot 50 \cdot 3 = 900$ classic problem instances. In addition to this, we followed a similar procedure to generate problem instances with constraints. Particularly, we generated 20 problem instances for each combination of problem types, classroom sizes (again 40, 60, and 80 students), proportion of students in compulsory student combinations (5 and 10%), and proportion of students in forbidden student combinations (5 and 10%). In this case, team sizes can range from 4 to 6 students. Thus, we generated $6 \cdot 20 \cdot 3 \cdot 2 \cdot 2 = 1,440$ problem instances with constraints.

We employed these classroom sizes as they are usual classroom sizes in most higher education courses in Spain. Despite the fact that some degrees may have cohorts of more than 80 students, it is typical to divide such larger cohorts into different subgroups that attend different lecture, seminar and laboratory sessions. Hence, we expect that, in practice, the algorithm will typically be employed in these subgroups whose sizes tend to be similar to the ones employed in these experiments. We acknowledge that, for larger problem instances it may be necessary to employ a different configuration of the proposed algorithm to achieve better results (e.g., population size, mutation rates, computation time, etc.) due to the increase in the size of the search space.

5.2. Baseline algorithms

To compare and analyze the performance of our proposed algorithm, we need to select a subset of algorithms from the state of the art. Exact methods like the ones proposed in [6] are discarded as they have been shown to scale poorly with the size of the classroom, and they would not be able to provide fast responses for many scenarios. Thus, we mainly focused on metaheuristic algorithms proposed in the literature. As many of the metaheuristic algorithms proposed in the literature have only been tested on a specific type of team formation in classroom problems, it makes sense to include in the comparison the metaheuristic algorithms proposed for solving the problem types described in the previous subsection. In addition to this, we also include a simple metaheuristic to act as a baseline for the other methods. Next, we briefly describe each of the metaheuristics included in the experiments.

- **Deterministic Crowding Genetic Algorithm (DCGA):** This is the metaheuristic proposed in [10], which was proposed to solve BRHNS problem types. It is based on a genetic algorithm employing a permutation chromosome representation, order crossover, swap mutation, tournament selection, and deterministic crowding to maintain diversity. As described in the article, the probability of crossover is 80%, the probability of mutating a solution is 5%, the population size is 100, and 50% of the population is selected to undergo the genetic operations.
- **Index Genetic Algorithm (IGA):** This is the metaheuristic proposed in [7,29] to solve BRGH problems. It uses an array chromosome representation where each position represents a student, and the content of the array position represents the index of the team it is assigned to. Moreover, it employs tournament selection, one-point crossover, swap mutation, and an elitist population management strategy. As described in the article, the probability of crossover is 80%, the probability of mutating a solution is 20%, the population size is 50, and half the population undergoes genetic operations.
- **Matrix Genetic Algorithm (MGA):** This metaheuristic, proposed in [8] and later employed in [9], employs a matrix chromosome representation, where each row represents a desired team and each column represents a prospective student. The authors propose to use the C1 crossover operator [46], swap mutation, and fitness proportionate selection. As described in the article, the population size is 80, 60% of the solutions are applied crossover, the probability of mutation is 20%, and the probability of mutating a gene is 15%.
- **Grasp (G):** We also included in the experiments a basic grasp algorithm using a permutation representation and a randomized greedy heuristic that selects a random team partition structure and then allocates students greedily. Then, local search is employed using the first improving strategy [42]. This method is included as a baseline for comparison with a standard and off-the-shelf metaheuristic. There are no hyperparameters for this metaheuristic.

Initially, these methods were proposed to deal with classic instances, as they often assume a fixed number of teams and no compulsory or forbidden combinations of students. We have to adapt the aforementioned methods slightly to use these algorithms in our second experiment, which involves instances with constraints. More specifically, the first three methods employ a heuristic in the second experiment to select the team partition structure. They assume that all solutions have the same team partition structure: one where all the teams have the same size except for, perhaps, the last team. The size selected aims to be as large as possible. In addition, invalid solutions are penalized with a large and negative fitness as described in Eq. (2). All algorithms, including our proposal, have been implemented from scratch in Python.

5.3. Experiment I: Classic problem instances

As mentioned, we compare our metaheuristic proposal with other state-of-the-art methods using classic problem instances in the first batch of experiments. That is, instances where only one team partition structure is valid and where there are no compulsory or forbidden combinations of students. Many state-of-the-art methods have been designed to work in this setting, but our method is not specifically designed for these scenarios. These experiments aim to show that, despite being designed to solve problems with constraints, our method can perform well in this setting.

We employed the hyperparameter values reported in their respective articles for all state-of-the-art algorithms described in the previous subsection. For our metaheuristic, we experimented with Bayesian optimization using the Optuna framework [47] to find a well-performing set of hyperparameters. To avoid overfitting all problem instances, we only employed one instance per type of problem instance (i.e., 6 instances in total). After the experiment, we fixed the hyperparameters of our method to a population size of $pop_{size} = 50$. As for mutation, a total of $n_{mut} = 5$ swap operations to simulate during the mutation was chosen. Regarding tournament selection, it selects $\alpha = 10\%$ of the individuals, with each tournament selecting $\beta = 30\%$ of the solutions and a $p_t = 80\%$ probability of selecting the winner of each tournament. The probabilistic crowding was configured with a $p_{pc} = 80\%$ probability of selecting the fittest among parent and child. Finally, the local search

Table 2
Average ranks for each problem type and the p-value when comparing the difference between the ranks, taking our algorithm as a control method.

	BRH						BRHNS					
	40		60		80		40		60		80	
	Avg. rank	p-value	Avg. rank	p-value	Avg. rank	p-value	Avg. rank	p-value	Avg. rank	p-value	Avg. rank	p-value
Ours	1.77	-	**1.17**	-	**1.02**	-	1.79	-	**1.14**	-	**1.01**	-
DCGA	1.85	0.83	1.91	0.02	1.98	2e-03	1.91	0.73	1.90	0.01	1.99	2e-03
IGA	5.00	0.00	5.00	0.00	5.00	0.00	5.00	0.00	5.00	0.00	5.00	0.00
MGA	3.94	1.4e-11	4.00	0.00	4.00	0.00	3.88	8.0e-11	4.00	0.00	4.00	0.00
G	2.44	0.04	2.92	4.3e-08	3.00	5.3e-10	2.42	0.06	2.96	1.2e-08	3.00	4.3e-10
	BRGH						**MBH**					
	40		60		80		40		60		80	
	Avg. rank	p-value	Avg. rank	p-value	Avg. rank	p-value	Avg. rank	p-value	Avg. rank	p-value	Avg. rank	p-value
Ours	1.42	-	1.26	-	**1.00**	-	2.18	-	1.85	-	1.59	-
DCGA	1.60	0.59	1.74	0.13	2.00	1.6e-03	2.18	1.00	1.89	0.93	1.83	0.46
IGA	4.76	0.00	4.20	0.00	4.06	0.00	5.00	0.00	5.00	0.00	5.00	0..
MGA	2.98	1.1e-06	3.00	5.2e-08	3.04	1.5e-10	3.43	1e-04	4.00	2.1e-11	4.00	5.2e-14
G	4.24	0.00	4.80	0.00	4.90	0.00	2.21	1.00	2.26	0.27	2.58	2.4e-03
	HTC4						**HTC8**					
	40		60		80		40		60		80	
	Avg. rank	p-value	Avg. rank	p-value	Avg. rank	p-value	Avg. rank	p-value	Avg. rank	p-value	Avg. rank	p-value
Ours	**1.00**	-	**1.00**	-	**1.00**	-	**1.00**	-	**1.00**	-	1.00	-
DCGA	2.00	1.6e-03	2.00	1.6e-03	2.00	1.6e-03	2.00	1.6-e03	2.00	1.6e-03	2.00	1.6e-03
IGA	5.00	0.00	4.74	0.00	4.32	0.00	4.98	0.00	4.64	0.00	4.36	0.00
MGA	4.00	0.00	4.26	0.00	4.68	0.00	4.02	0.00	4.36	0.00	4.64	0.00
G	3.00	3.5e-10	3.00	3.5e-10	3.00	3.5e-10	3.00	3.5-10	3.00	3.5-10	3.00	3.5e-10

was configured to select $\gamma = 10\%$ of the fittest individuals in the population for roulette wheel selection and then, on the selected individual, carry out a maximum of $n_{ls} = 20$ local improvement steps. The local search is applied to each generation with a $p_{ls} = 30\%$ probability.

To test all the methods, each algorithm solved each problem instance 30 times. Each algorithm run is limited to 3 s to compare all of the metaheuristics in a fair setting. The best result obtained by the metaheuristic is collected at the end of the deadline. The deadline of 3 s was decided to understand the performance of the methods in a real-time environment like a user application. Then, for each combination of algorithm and specific problem instance, we compute the average value of the objective function to calculate the average performance of each algorithm for each problem instance.

After that, we calculate the rank achieved by each algorithm in each problem instance, with lower ranks corresponding to higher average values of the objective function. As recommended by [48] and [49], we carried out a multiproblem analysis for each family of problems and classroom size. In this case, as all the problem instances are employed, each multiproblem analysis comprises the results of 50 problem instances. To compare the results of each algorithm, we carried out a non-parametric Friedman test ($\alpha = 0.05$) for each family of problems to detect any potential differences between the ranks achieved by the metaheuristics. If the test detects any statistical difference, we conduct a post-hoc analysis, taking our algorithm as a control method and comparing its performance against each crossover operator. Particularly, and following the recommendations for comparing one control method against other methods in evolutionary computing [49], we consider that the difference between rankings is explained by a standard normal distribution whose standardization follows the next expression:

$$z = \frac{R_i - R_j}{\sqrt{\frac{k(k+1)}{6n}}} \tag{6}$$

where R_i is the average ranking achieved by the ith crossover operator, k is the number of algorithms in the experiment, and n is the number of problem instances. Then, the p-value of observing a value at least as extreme as z is calculated. To adjust for the family-wise error rate, we adjust the obtained p-values employing the Holm-Bonferroni correction. All experiments were executed on an AMD Ryzen 5 2600 with 6 cores of 3.4 GHz and 16 GB of RAM.

The results for the first experiment can be observed in Table 2. The table shows the average rank of each algorithm for each problem type employed in the experiment. Next to the average rank, one can find the p-value of the pairwise comparison between the average rank obtained by our algorithm and another algorithm. In addition, we have shadowed in green those problem types where our proposal obtained a better rank, which is statistically better than any other algorithm.

The first thing that can be observed is that the average rank obtained by our proposed method is never worse than the one obtained by other algorithms. It is strictly higher in 17 out of the 18 problem types, and the test has enough power to find statistical differences in 10 out of the 18 problem types. This result supports our initial aims, designing an algorithm whose performance is robust to different problem types and objective functions (goal iii). The results also suggest that despite being designed for the specific case of problems with constraints, our method achieves remarkable performance for classic problem instances, which was another of our aims (goal i).

Our method did not achieve statistically different results for MBH problem instances. In this case, we found that our method, DCGA, and the grasp method converged to the same solution most of the time. This probably indicates that they could converge to the optimal value and that these problem types are less complex than others employed in this study. While this happens in smaller problem instances, the differences increase in favor of our method as one deals with larger problem instances.

This same trend is also observed for other types of problems. Our algorithm does not have larger populations than other algorithms in the study, except for the Grasp algorithm, which is not based on a population of solutions. This suggests that our algorithm implements better search strategies and heuristics than other algorithms employed in the study. Hence, it seems that it is better adapted to deal with larger problem instances, supporting one of our initial goals of developing a method that is also robust to the problem size (goal iv).

Our method has remarkable performance for the HTC4 and HTC8 problems, where our method was always the best choice for all the problem instances tested. In this type of problem instance, one aims to calculate team partitions where teams are similar and homogeneous in their skills and characteristics. We argue that this is a common criterion when forming teams in the classroom, as lecturers typically want to avoid unbalanced teams. The results achieved by our algorithm for this type of problem show the applicability of our algorithm in real team formation in classroom scenarios.

Moreover, it should also be highlighted that our algorithm can achieve better performance than methods specifically proposed for solving some specific types of problems. For instance, DCGA was initially proposed to solve BRHNS instances, IGA was proposed for BRGH instances, and MGA was proposed for instances similar to HTC4 and

HTC8. In all of those instances, the general trend is that our algorithm outperforms other methods, especially for larger instances.

Another interesting observation stems from the performance of the grasp method. Despite being the implementation of a simple metaheuristic, it is capable of achieving the third position in almost any problem instance. Typically, many studies on team formation in the classroom have opted for population-based metaheuristics like genetic algorithms and have overlooked other types of metaheuristics like the ones based on local search or constructive metaheuristics. We argue that the results obtained in this study highlight the prospective potential that other types of metaheuristics may have on team formation in the classroom problem. One could argue that our proposal has many elements from constructive and local search metaheuristics despite being also based on a population of solutions.

5.4. Experiment II: Instances with constraints

In this second experiment, we aim to compare our proposed algorithm with problems with different valid team sizes and constraints in the form of compulsory and forbidden student combinations. The specific adaptions on the state-of-the-art methods can be read in Section 5.2. For the configuration of our proposed algorithm, we employed again Bayesian optimization to find good hyperparameters. The hyperparameters were optimized using 1 problem instance per type (6 in total). The results were similar to the ones in the previous experiments, so we kept all the hyperparameter values the same. The only exceptions are the hyperparameters related to the team partition structure, which were not relevant in the previous experiment (as there was only one valid team partition structure) and are relevant in this experiment. The probability of inheriting the best team partition structure between pairs of solutions is set to $p_n = 40\%$, and the probability of mutating a team partition structure is $p_{mt} = 30\%$. The parameter that controls the exponential ranking selection employed to mutate team partition structures is set to $c = 0.1$, which will tend to select very similar team partition structures.

We employ the same experimental conditions described in the previous experiments (i.e., number of repetitions, deadline, etc.). In addition, we carry out the same type of analysis based on rankings, the Friedman test, and the posthoc analysis based on the differences between rankings. Next, we outline the main highlights that can be extracted from the second batch of experiments using a multiproblem analysis. The results of the experiment can be found in Table 3. The table shows the average rank obtained by each algorithm for each combination of problem type and problem size. Thus, each multiproblem analysis consists of the analysis of 80 problem instances. Below the average rank, one can find the percentage of the algorithm runs that finished with a feasible solution (i.e., no constraints violated). Next to the average rank, one can find the p-value of the pairwise comparison between the average rank obtained by our algorithm and another algorithm. In addition, we have shadowed in green those problem types where our proposal obtained a better rank and is statistically better than any other algorithm.

The results suggest that the differences between our method and the state-of-the-art methods tend to be larger. Our algorithm is now statistically better than other algorithms in 16 out of the 18 types of problem instances, and it is no worse than the other methods in the other 2 types of problem instances. Our method does not achieve statistically different results only for MBH problems with 40 and 60 students. As we observed in the previous experiments, these problem instances seem easier to solve. These results support our goal of designing an algorithm capable of performing well in team formation in classroom problems with constraints (goal ii) and designing a robust algorithm independent of the type of objective function (goal iii).

In terms of feasibility, our algorithm could find a feasible solution for every possible run. At the same time, the performance of the other methods tends to deteriorate for most types of problems as the size of the problem increases. Therefore, our algorithm was also robust regarding the problem size, supporting one of our goals (goal iv). The IGA and MGA algorithms were especially sensitive to more constraints and larger problem sizes, while the DCGA method was not as affected as the other two. However, it could not achieve as good fitness as the one obtained by our algorithm. The main difference between DCGA, IGA, and MGA is the representation employed for solutions. Similarly to our solution, DCGA employs a permutation representation, IGA employs a vector of indices, and MGA employs a matrix representation. The results suggest that permutation representations and their associated operators may be more apt for problems with constraints than the other representations, given the large differences in the results obtained in this experiment.

As in the previous experiment, the performance of the Grasp algorithm is remarkable for most types of problems. This result points out again the promising research direction of exploring other types of metaheuristics based on trajectories and local search for team formation problems in the classroom.

5.5. Experiment III: Analyzing the computation time

Finally, we designed some experiments to analyze how our proposed metaheuristic behaves with respect to the computation time. More specifically, we designed two sets of experiments for this matter. The first set of experiments analyzes how the computation time of our metaheuristic changes along with the number of students for a fixed number of iterations. By setting the same number of iterations for different numbers of students, one can understand how the computation time per iteration changes as the problem size increases. The second set of experiments aims to determine how the computation time is increased until a good solution is found, considering that the search space increases with the number of students and, thus, problems become harder to solve.

For the first experiments, we employed the metaheuristic configuration found by Bayesian optimization and reported in previous experiments. For this experiment, we used the problem instances with constraints employed in Section 5.4, as well as the problem instances with 20 students generated by the same procedure described in Section 5.1. This gives a total of $6{\cdot}20{\cdot}4{\cdot}2{\cdot}2 = 1,920$ problem instances. Each problem instance was solved once using our metaheuristic by computing 300 iterations, and then we annotated the average computation time per iteration for each algorithm run. It should be remembered that the goal is to understand how the computation time per iteration increases with the number of students.

The results of this experiment can be observed in Fig. 3. The graphs show the average computation time per iteration, in seconds, of the proposed metaheuristic running for 300 iterations and its confidence interval at 95% for each problem type, number of students, and constraint configuration. $|C|$ denotes the percentage of students in compulsory combinations, and $|N|$ denotes the percentage of students in forbidden combinations. Firstly, it should be highlighted that the computation time per iteration generally increases with the number of students, as we expected. This is reasonable since many operators employed in the proposed metaheuristic increase its computation with more students. Secondly, it should be highlighted that the differences in average computation time between different problem instances can be attributed to how the fitness is computed for these problem instances since the logic of the proposed metaheuristic is the same regardless of the problem type. BRGH, HTC4, and HTC8 tend to have more complex fitness functions and, therefore, higher computation times. Thirdly, the lines suggest that the computation time per iteration scales reasonably with the number of students regardless of the problem type and the constraint configuration. We can analyze the increase in the average total computation time between the smallest and the largest problem instances (4 times the size of the smallest problem instances) to assess the growth of the computation time. For example,

Table 3
Results for the instances with constraints. Average ranks per method for each problem instance type, the percentage of runs that found a feasible solution, and the *p*-value when comparing our algorithm with respect to the other algorithms.

	BRH						BRHNS					
	40		60		80		40		60		80	
	Avg. rank	p-value	Avg. rank	p-value	Avg. rank	p-value	Avg. rank	p-value	Avg. rank	p-value	Avg. rank	p-value
Ours	**1.56** **100%**	-	**1.18** **100%**	-	**1.01** **100%**	-	**1.44** **100%**	-	1.11 100%	-	1.00 100%	-
DCGA	2.38 100%	1.2e-03	2.4 99.5%	3.9e-08	3.16 86.8%	0.00	2.31 99.9	5.3e-04	2.12 99.3%	5.3e-05	3.10 86.9%	0.00
IGA	4.90 72.1%	0.00	5.00 17.3%	0.00	4.92 11.6%	0.00	4.97 66.3%	0.00	5.00 16.8%	0.00	4.83 20.7%	0.00
MGA	3.67 100%	0.00	3.83 98.2%	0.00	3.82 57.4%	0.00	3.75 100%	0.00	3.93 96.8%	0.00	4.06 55.41%	0.00
G	2.47 100%	4.0e-04	2.57 100%	1.2e-06	2.05 100%	3.4e-05	2.51 100%	2.3e-05	2.82 100%	1.0e-11	2.00 86.9%	6.5e-05
	BRGH						**MBH**					
	40		60		80		40		60		80	
	Avg. rank	p-value	Avg. rank	p-value	Avg. rank	p-value	Avg. rank	p-value	Avg. rank	p-value	Avg. rank	p-value
Ours	**1.00** **100%**	-	**1.00** **100%**	-	**1.00** **100%**	-	1.93 100%	-	1.68 100%	-	**1.00** **100%**	-
DCGA	2.00 100%	6.5e-05	2.00 99.8%	6.5e-05	2.08 85.3%	1.4e-05	2.15 100%	0.41	1.68 99.7%	1.00	3.13 87.5%	0.00
IGA	4.65 82.9%	0.00	5.00 36.3%	0.00	4.32 21.3%	0.00	3.64 100%	1.8e-11	5.00 97.4%	0.00	4.80 54.0%	0.00
MGA	3.00 100%	1.8e-15	3.22 91.3%	0.00	3.85 47.4%	0.00	5.00 73.4%	0.00	3.94 21.5%	0.00	4.05 19.7%	0.00
G	4.35 100%	0.00	3.77 92.7%	\0.00	3.73 49.7%	0.00	2.26 100%	0.25	2.70 100%	6.3e-05	2.00 100%	6.5e-05
	HTC4						**HTC8**					
	40		60		80		40		60		80	
	Avg. rank	p-value	Avg. rank	p-value	Avg. rank	p-value	Avg. rank	p-value	Avg. rank	p-value	Avg. rank	p-value
Ours	**1.00** **100%**	-	**1.00** **100%**	-	**1.00** **100%**	-	**1.00** **100%**	-	**1.00** **100%**	-	**1.00** **100%**	-
DCGA	2.02 99.9%	4.3e-05	2.05 99.8%	2.7e-05	2.58 90.5%	2.9e-10	2.00 100%	6.5e-05	2.03 99.8%	3.4e-05	2.58 90.7%	2.9e-10
IGA	4.98 74.1%	0.00	5.00 22.6%	0.00	4.63 23.1%	0.00	4.96 74.1%	0.00	5.00 22.7%	0.00	4.66 23.4%	0.00
MGA	4.00 100%	0.00	3.98 97.2%	0.00	4.36 46.4%	0.00	4.03 100%	0.00	4.00 95.6%	0.00	4.31 47.1%	0.00
G	2.98 100%	2.4e-15	2.96 100%	5.8e-15	2.41 96.9%	1.6e-08	3.00 100%	1.8e-15	2.96 100%	5.8e-15	2.42 99.0%	1.2e-08

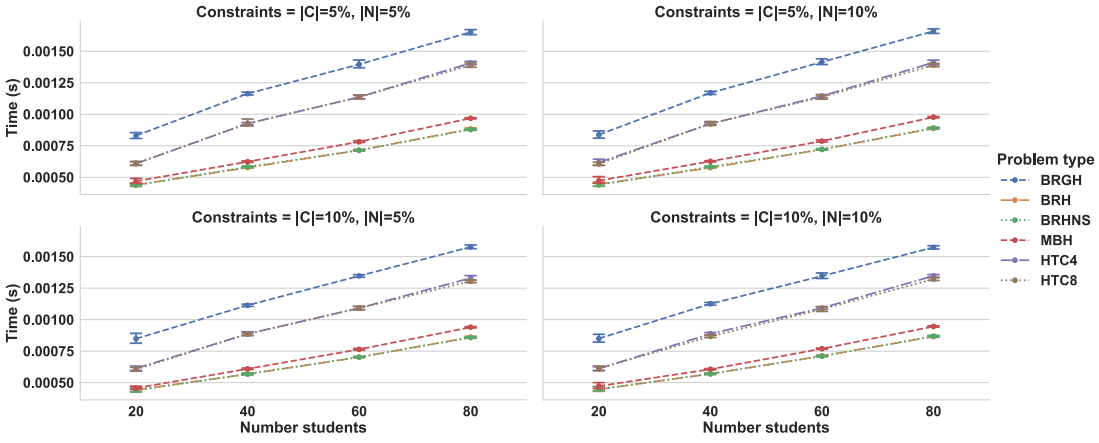


Fig. 3. Average computation time per iteration for the proposed metaheuristic for different problem types, number of students, and constraint configuration.

for cases with $|C| = 5\%$ and $|N| = 5\%$ one can realize that the average computation time only increased by 0.00140/0.00061=2.30 times and 0.00139/0.00061=2.27 times for HTC4 and HTC8 instances, which are the ones that show higher increases in the computation time. Even though the number of students quadrupled, the average computation time per iteration only roughly doubled. Similar ratios were found for other configurations of $|C|$ and $|N|$. Therefore, it can be concluded that the computation time per iteration scales well with increases in the number of students.

For the second experiment, we changed the stop criteria of the metaheuristic. The algorithm stops its execution when the best solution in the population does not improve for 100 iterations. We consider that if the algorithm has not been capable of improving the best solution in 100 iterations, it has found a good solution. Our metaheuristic solved each problem instance once, and we annotated the total computation time in seconds. In this case, Fig. 4 shows the results of this experiment by portraying the average computation time for each combination of problem type, number of students, and constraint configuration.

Again, the differences in average computation time between different problem instances can be attributed to how the fitness is computed. Similarly to the previous experiment, one can observe that the total time of the proposed metaheuristic scales reasonably with the

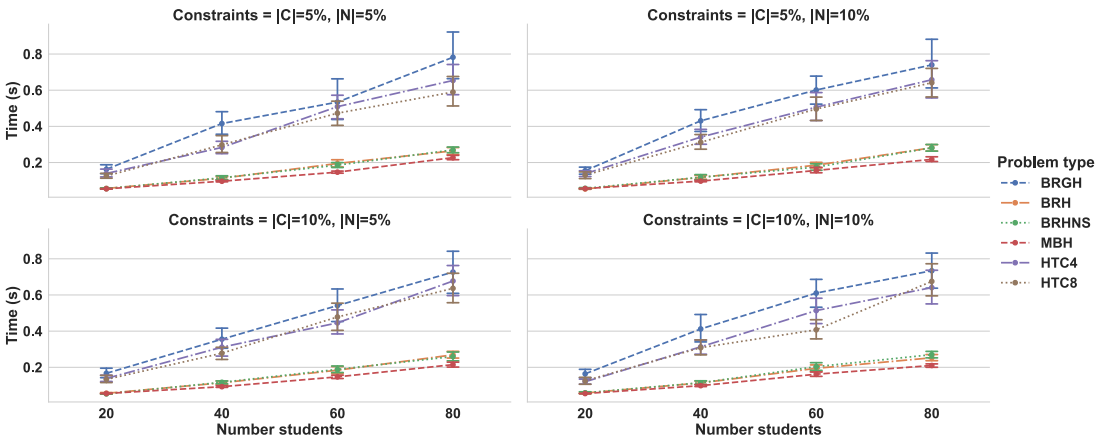


Fig. 4. Average computation time for the proposed metaheuristic stopping when the best solution has not improved in 100 iterations.

number of students for all problem types. Doing a similar analysis, we compare the increase in the average computation time of the largest instances with respect to the smallest instances. For instance, for $|C| = 5\%$ and $|N| = 5\%$ the increase is of 0.782/0.162=4.82 for BRGH, 0.227/0.055=4.13 for MBH, 0.267/0.056=4.78 for BRHNS, 0.265/0.057=4.65 for BRH, 0.654/0.139=4.70 for HTC4, and 0.590/0.126=4.68 for HTC8. That is, the average computation time increased in roughly the same order as the number of students. Similar ratios were found for other configurations of $|C|$ and $|N|$. In addition to this, it can be observed that, on average, the algorithm converges to a good solution in less than one second.

As mentioned, these classroom sizes are usual in many higher education settings in Spain. If the proposed metaheuristics is to be employed for larger problem instances it may be necessary to adjust some of the parameters of the algorithm to account for the larger search space. Nevertheless, the experiments shown in this section suggest that the average computation time increases in a controlled way with the number of students. In fact, even for classroom sizes of 80 students the average computation time did not reach one second. These results suggest that the proposed metaheuristic can be employed for many realistic classroom settings (goal iv).

6. Conclusions

Team formation in the classroom is a crucial topic for developing teamwork-related skills in the classroom. The scientific community has invested considerable effort in proposing various solutions for automating team formation. Specifically, the use of metaheuristics is quite common for solving these types of problems due to the high combinatorial complexity, which makes applying exact methods inappropriate for reasonable response times as the size of the problem increases.

Our approach to team formation in the classroom problem allows for incorporating constraints on flexible team sizes during team formation, defining constraints on students who must be together in the same team, and defining constraints for students who cannot be together on the same team. Despite being realistic as they capture human relationships in the classroom, these constraints are missing in many state-of-the-art approaches.

Given this setting, we propose a general evolutionary algorithm that combines mutation, the estimation of synergy between team members, and local search. The proposed metaheuristic is independent of the criteria that guide the team formation process. It does not assume anything about the objective function and treats it as a black box. Instead, we guide the decisions made by our evolutionary algorithm on the general concept of synergy between team members to estimate the quality of teams. This has allowed us to evaluate the performance of our proposal with different types of objective functions used to assess the quality of a team partition.

According to our experiments, our proposal outperforms other state-of-the-art metaheuristics used in this paper for classic problem instances, i.e., problems without constraints. Specifically, our proposal achieved the best average rank in 17 out of 18 types of problems, with significant differences in 10 of them. It was never worse than any other algorithm. These results validate that, despite being designed for scenarios with constraints, our algorithm still outperforms state-of-the-art algorithms. In addition to this, this performance is robust to different types of problems and objective functions. Moreover, our proposal has better scalability as instance sizes grow than other state-of-the-art algorithms. In experiments where constraints related to team size and compulsory and forbidden combinations of students, the differences between our proposal and the rest of the state-of-the-art algorithms were even greater than in the previous experiment, achieving statistically better performance in 16 out of 18 types of problems and it was never worse than other algorithms. Therefore, our algorithm performs well in scenarios requiring constraints related to valid team sizes and the existence of compulsory and forbidden combinations of students.

In addition, we conducted experiments to analyze the computation time of the proposed metaheuristic as the number of students in the classroom increases. The experiments suggest that the average computation time per iteration and the total time until convergence to a good solution scales well with the number of students in the classroom. These results suggest that the proposed metaheuristic can be employed in realistic classroom settings for a wide variety of scenarios.

Despite these positive results, this present work has some limitations. Firstly, the proposed algorithm assumes that team performance can be approximated via the calculation of the synergies between its team members. Particularly, our current implementation approximates team performance based on the synergy between pair of students. While this may be a good approximation in many settings, as shown in the experiments, there may be objective functions where interactions between more team members need to be considered or where the synergy between two team members is not a good base for approximating the overall team performance. In those cases, some of the decisions made by our algorithm during the search process may not be as accurate as in the domains employed in this work. Secondly, our proposed algorithm has been tested to work with usual classroom sizes in higher education. If the proposed algorithm is to be employed with much larger classroom sizes it may be necessary to explore other parameters configurations (e.g., population size, mutation rate, etc.) to cope with the larger search space or one may need to divide the classroom into different subgroups and then apply the proposed algorithm to each subgroup. Nevertheless, we argue that the classroom sizes employed in the experiments are usual in many settings. Finally, while we tested our proposed algorithm

in a wide variety of problem types and showed its good performance in those, it is acknowledged that, as it is typical in optimization, no metaheuristic is superior in every single problem instance. We focused on testing our algorithm for those problem domains that we could reproduce with our available data, problems whose data is available in the research field or can be easily generated. However, there may be problem instances where our proposed algorithm is not the best performing algorithm.

In conclusion, our algorithm is a good choice for team formation in classroom problems with or without constraints. As a future line of work, we plan to study incorporating adaptive mechanisms and reinforcement learning during the search process to improve the performance of current metaheuristic algorithms for the team formation problem. In addition to this, we also plan to study new metaheuristics based on local search, which obtained promising results in our experiments.

CRediT authorship contribution statement

Gonzalo Candel: Software, Conceptualization. **Victor Sánchez-Anguix:** Writing – review & editing, Writing – original draft, Validation, Supervision, Software, Methodology, Investigation, Data curation, Conceptualization. **Juan M. Alberola:** Supervision, Software, Investigation, Conceptualization. **Vicente Julián:** Writing – review & editing, Supervision, Project administration, Investigation, Funding acquisition, Conceptualization. **Vicent Botti:** Writing – review & editing, Supervision, Funding acquisition, Conceptualization.

Declaration of competing interest

The authors declare the following financial interests/personal relationships which may be considered as potential competing interests: Victor Sanchez-Anguix reports financial support was provided by Spain Ministry of Science and Innovation. Vicente Julian reports financial support was provided by Spain Ministry of Science and Innovation. Vicent Botti reports financial support was provided by European Commission. The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper. If there are other authors, they declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This work was partially supported by MINECO/FEDER RTI2018-095390-B-C31 project of the Spanish government, project TED2021-131295B-C32 from the State Research Agency, and DIGITAL2022 CLOUDAI02/S8760000 from the European Commission, . Some of the authors are partially supported by the Spanish Ministry of Science and Innovation under the project PID2021-123673OB-C31 COSASS and project "OPRES-Realistic Optimization in Problems in Public Health" (No. PID2021-124975OB-I00), partially financed with FEDER funds. Additional support is provided by project INNVA1/2024/91 funded by Agencia Valenciana de la Innovación.

Data availability

Data will be made available on request.

References

[1] E.D. Prada, M. Mareque, M. Pino-Juste, Teamwork skills in higher education: is university training contributing to their mastery? Psicologia: Reflexãoe Crítica 35 (2022).
[2] C. Mellon, What are the Benefits of Group Work?, Compiled by Dr. Chapman, 2015.
[3] D. Thompson, I. Anitsal, H. Barrett, Attitudes toward teamwork in higher education: a comparative study of religiously affiliated universities and secular-based universities, in: Allied Academies International Conference. Academy of Educational Leadership. Proceedings, Vol. 13, No. 2, Jordan Whitney Enterprises, Inc, 2008, p. 86.
[4] J.M. Alberola, E. Del Val, V. Sanchez-Anguix, A. Palomares, M.D. Teruel, An artificial intelligence tool for heterogeneous team formation in the classroom, Knowl.-Based Syst. 101 (2016) 1–14.
[5] J. Juárez, C. Santos, C.A. Brizuela, A comprehensive review and a taxonomy proposal of team formation problems, ACM Comput. Surv. 54 (7) (2021) 1–33.
[6] G. Candel, V. Sánchez-Anguix, J.M. Alberola, V. Julián, V. Botti, An integer linear programming model for team formation in the classroom with constraints, in: International Conference on Hybrid Artificial Intelligence Systems, Springer, 2023, pp. 397–408.
[7] N. Ugarte, A. Aranzabal, A. Arruarte, M. Larrañaga, Using the behavioural tendency of students in a team environment for team formation, in: 2022 IEEE Frontiers in Education Conference, FIE, IEEE, 2022, pp. 1–9.
[8] J. Moreno, D.A. Ovalle, R.M. Vicari, A genetic algorithm approach for group formation in collaborative learning considering multiple student characteristics, Comput. Educ. 58 (1) (2012) 560–569.
[9] O.R. Sánchez, C.A.C. Ordonez, M.A.R. Duque, I.I.B.S. Pinto, Homogeneous group formation in collaborative learning scenarios: An approach based on personality traits and genetic algorithms, IEEE Trans. Learn. Technol. 14 (4) (2021) 486–499.
[10] V. Yannibelli, A. Amandi, A deterministic crowding evolutionary algorithm to form learning teams in a collaborative learning context, Expert Syst. Appl. 39 (10) (2012) 8584–8592.
[11] E. Andrejczuk, F. Bistaffa, C. Blum, J.A. Rodríguez-Aguilar, C. Sierra, Synergistic team composition: A computational approach to foster diversity in teams, Knowl.-Based Syst. 182 (2019) 104799.
[12] J.-M. Flores-Parra, M. Castañón-Puga, R.D. Evans, R. Rosales-Cisneros, C. Gaxiola-Pacheco, Towards team formation using belbin role types and a social networks analysis approach, in: 2018 IEEE Technology and Engineering Management Conference, TEMSCON, IEEE, 2018, pp. 1–6.
[13] L. Sanz-Martínez, A. Martínez-Monés, M.L. Bote-Lorenzo, J.A. Munoz-Cristóbal, Y. Dimitriadis, Automatic group formation in a MOOC based on students' activity criteria, in: European Conference on Technology Enhanced Learning, Springer, 2017, pp. 179–193.
[14] R.M. Belbin, Team Roles at Work, Routledge, 2012.
[15] V. Sanchez-Anguix, J.M. Alberola, E. Del Val, A. Palomares, M.D. Teruel, Comparing computational algorithms for team formation in the classroom: a classroom experience, Appl. Intell. 53 (20) (2023) 23883–23904.
[16] I.B. Myers, The Myers-Briggs Type Indicator: Manual, Consulting Psychologists Press, 1962.
[17] O.P. John, E.M. Donahue, R.L. Kentle, Big five inventory, J. Pers. Soc. Psychol. (1991).
[18] A. Anagnostopoulos, L. Becchetti, C. Castillo, A. Gionis, S. Leonardi, Online team formation in social networks, in: Proceedings of the 21st International Conference on World Wide Web, 2012, pp. 839–848.
[19] J.H. Gutiérrez, C.A. Astudillo, P. Ballesteros-Pérez, D. Mora-Melià, A. Candia-Véjar, The multiple team formation problem using sociometry, Comput. Oper. Res. 75 (2016) 150–162.
[20] M. Fathian, M. Saei-Shahi, A. Makui, A new optimization model for reliable team formation problem considering experts' collaboration network, IEEE Trans. Eng. Manage. 64 (4) (2017) 586–593.
[21] C. Odo, J. Masthoff, N. Beacham, Group formation for collaborative learning: A systematic literature review, in: Artificial Intelligence in Education: 20th International Conference, AIED 2019, Chicago, IL, USA, June 25-29, 2019, Proceedings, Part II 20, Springer, 2019, pp. 206–212.
[22] N. Maqtary, A. Mohsen, K. Bechkoum, Group formation techniques in computer-supported collaborative learning: A systematic literature review, Technol. Knowl. Learn. 24 (2019) 169–190.
[23] T. Varvel, S.G. Adams, S.J. Pridie, B.C. Ruiz Ulloa, Team effectiveness and individual Myers-Briggs personality dimensions, J. Manag. Eng. 20 (4) (2004) 141–146.
[24] N.L. Quenk, Essentials of Myers-Briggs Type Indicator Assessment, John Wiley & Sons, 2009.

[25] H. Ma, J. Li, H. Zhu, W. Tang, Z. Huang, Y. Tang, Collaborative optimization of learning team formation based on multidimensional characteristics and constraints modeling: A team leader-centered approach via E-CARGO, IEEE Trans. Comput. Soc. Syst. (2022).
[26] B.E. de Raad, M.E. Perugini, Big Five Assessment, Hogrefe & Huber Publishers, 2002.
[27] Ž. Komlenov, Z. Budimac, M. Ivanović, K. Bothe, Team formation process automatization, in: EDUCON, 2013, pp. 744–750.
[28] S. Garshasbi, Y. Mohammadi, S. Graf, S. Garshasbi, J. Shen, Optimal learning group formation: A multi-objective heuristic search strategy for enhancing inter-group homogeneity and intra-group heterogeneity, Expert Syst. Appl. 118 (2019) 506–521.
[29] A. Aranzabal, E. Epelde, M. Artetxe, Team formation on the basis of Belbin's roles to enhance students' performance in project based learning, Educ. Chem. Eng. 38 (2022) 22–37.
[30] S. Luke, Essentials of metaheuristics, 2009.
[31] Z. Li, X. Lin, Q. Zhang, H. Liu, Evolution strategies for continuous optimization: A survey of the state-of-the-art, Swarm Evol. Comput. 56 (2020) 100694.
[32] F. Neri, C. Cotta, P. Moscato, Handbook of Memetic Algorithms, vol. 379, Springer, 2011.
[33] J.T. Scarnati, On becoming a team player, Team Perform. Management: An Int. J. 7 (1/2) (2001) 5–10.
[34] Y.-Y. Leu, L.A. Matheson, L.P. Rees, Assembly line balancing using genetic algorithms with heuristic-generated initial populations and multiple evaluation criteria, Decis. Sci. 25 (4) (1994) 581–605.
[35] B. Saavedra-Moreno, S. Salcedo-Sanz, A. Paniagua-Tineo, L. Prieto, A. Portilla-Figueras, Seeding evolutionary algorithms with heuristics for optimal wind turbines positioning in wind farms, Renew. Energy 36 (11) (2011) 2838–2844.
[36] P. Larranaga, C.M.H. Kuijpers, R.H. Murga, I. Inza, S. Dizdarevic, Genetic algorithms for the travelling salesman problem: A review of representations and operators, Artif. Intell. Rev. 13 (1999) 129–170.
[37] J.P. Contreras, P. Bosch, M. Varas, F. Basso, A new genetic algorithm encoding for coalition structure generation problems, Math. Probl. Eng. 2020 (2020) 1–13.
[38] T. Blickle, L. Thiele, A comparison of selection schemes used in evolutionary algorithms, Evol. Comput. 4 (4) (1996) 361–394.
[39] A.E. Eiben, J.E. Smith, Introduction to Evolutionary Computing, Springer, 2015.
[40] T. Blickle, Tournament selection, in: Evolutionary Computation 1: Basic Algorithms and Operators, CRC Press, 2018, p. 181.
[41] O.J. Mengshoel, D.E. Goldberg, The crowding approach to niching in genetic algorithms, Evol. Comput. 16 (3) (2008) 315–354.
[42] M.G. Resende, C.C. Ribeiro, Optimization by GRASP, Springer, 2016.
[43] R.M. Belbin, V. Brown, Team Roles at Work, Routledge, 2022.
[44] S.P. King, B.A. Mason, Myers-Briggs type indicator, in: The Wiley Encyclopedia of Personality and Individual Differences: Measurement and Assessment, Wiley Online Library, 2020, pp. 315–319.
[45] A. Ounnas, H. Davis, D. Millard, A framework for semantic group formation, in: 2008 Eighth IEEE International Conference on Advanced Learning Technologies, IEEE, 2008, pp. 34–38.
[46] C.R. Reeves, A genetic algorithm for flowshop sequencing, Comput. Oper. Res. 22 (1) (1995) 5–13.
[47] T. Akiba, S. Sano, T. Yanase, T. Ohta, M. Koyama, Optuna: A next-generation hyperparameter optimization framework, in: Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining, 2019, pp. 2623–2631.
[48] S. Garcıa, D. Molina, M. Lozano, F. Herrera, A study on the use of non-parametric tests for analyzing the evolutionary algorithms' behaviour: a case study on the CEC'2005 special session on real parameter optimization, J. Heuristics 15 (6) (2009) 617–644.
[49] J. Derrac, S. García, D. Molina, F. Herrera, A practical tutorial on the use of nonparametric statistical tests as a methodology for comparing evolutionary and swarm intelligence algorithms, Swarm Evol. Comput. 1 (1) (2011) 3–18.



Gonzalo Candel is a Data Scientist based in Valencia, Spain. Born in 2000, he developed a solid mathematical foundation early on, completing the Kumon programme in 2016. He graduated with a Bachelor's degree in Data Science in 2022 at Universitat Politècnica de València, culminating his studies with a thesis on optimization techniques, specifically focusing on the design of evolutionary algorithms for team formation. Since then, Gonzalo has gained valuable work experience in various domains, including time series forecasting, industrial applied optimization, and Big Data analytics.



Victor Sanchez-Anguix is currently an associate professor at Universitat Politècnica de València and a member of the Applied Optimization Systems (SOA) group at Instituto de Tecnología Informática. His research area focuses on artificial intelligence and applied optimization, with a special focus on heuristic-based automated negotiation between autonomous agents. He has also worked on the application of optimization algorithms in education, with a special focus on the operational management of educational institutions and classrooms. He has participated in more than 10 research projects, both national and international, in the areas of artificial intelligence and optimization.



Juan Miguel Alberola holds a Ph.D. in Artificial Intelligence and is an Associate Professor at the Universitat Politècnica de València (UPV). He also leads the Smart AI Industry Chair in collaboration with Telefónica, focusing on smart cities. He is a member of the GTI-IA research group and part of the VRAIN Institute. Dr. Alberola has participated in over 20 research projects and published more than 20 articles in JCR-indexed journals. His research areas include multi-agent systems, mobility, optimization, and intelligent assistants.



Vicente Julian holds a position of Full Professor of Computer Science at the Universitat Politècnica de València (UPV) where he has taught since 1996. He is member of the Valencian Research Institute for Artificial Intelligence (VRAIN), and Coordinator of the Computer Science PhD Program at the UPV. Four international projects, two international excellence networks, twenty-five Spanish projects and four technology transfer projects have covered the research on Artificial Intelligence. He has more than 300 works published in journals with outstanding positions in the JCR or published in relevant conference proceedings. Vicente Julian has supervised 10 Ph.D. Thesis.



Vicent J. Botti is Full Professor of Computer Sciences at the Universitat Politècnica de València where he has been teaching since 1984. He is Director of the Valencian Research Institute for Artificial Intelligence (VRAIN) with more than 90 researchers in AI and of the Grupo de Tecnología Informática – Inteligencia Artificial (GTI-IA). He has more than 370 publications in international journals and conferences. He has participated in 64 research projects, including 6 EU projects, and more than 30 national projects, being the principal researcher in 26, and 21 contracts with companies. He has supervised 24 Ph.D. Thesis.